

รายงาน Lab 9 Spring Boot Transaction

รายวิชา: CP353002 Principles of Software Design

เรื่อง: ระบบฝากเงินและการจัดการ Transaction ด้วย Spring Boot, JPA และ PostgreSQL

ชื่อ-นามสกุล: นายปวัฒน์ ปัดมามา

รหัสนักศึกษา: 673380048-6

Section: 1

ส่วนที่ 1 หลักการออกแบบ

1.1 ภาพรวมของระบบ

ระบบฝากเงินเป็น REST API สำหรับสร้างบัญชี ดูข้อมูลบัญชี และฝากเงิน โดยบันทึกข้อมูลในฐานข้อมูล PostgreSQL ชื่อ lab9 ประกอบด้วยตาราง account และ deposit_transaction การฝากเงินหนึ่งครั้งต้องเพิ่มยอดในบัญชีและสร้างรายการประวัติให้สำเร็จไปด้วยกัน

1.2 การแบ่งหน้าที่ของแต่ละชั้น

AccountController รับ HTTP Request และเรียกใช้ Service ส่วน AccountService

จัดการการสร้างและค้นหาบัญชี และ DepositService จัดการขั้นตอนการฝากเงิน Repository

ติดต่อฐานข้อมูลผ่าน Spring Data JPA ขณะที่ Entity แทนข้อมูลของตารางแต่ละตัว

Controller รับ AccountService และ DepositService ผ่าน constructor เดียวกัน ส่วน Service รับ

Repository ผ่าน constructor และเก็บในฟิลด์ final รูปแบบนี้ทำให้เห็น Dependency

ของแต่ละคลาสชัดเจนและสามารถแยกทดสอบได้

1.3 ความสัมพันธ์แบบ One-to-Many

หนึ่ง Account มี DepositTransaction ได้หลายรายการ แต่รายการฝากแต่ละรายการอ้างอิงถึงบัญชีหนึ่งบัญชี Foreign Key อยู่ที่ฝั่งรายการฝาก ดังนี้

deposit_transaction.account_id → account.id

ในโค้ดกำหนด @ManyToOne และ @JoinColumn(name = "account_id") ที่ DepositTransaction เท่านั้น ส่วน Account ไม่มีลิสต์รายการฝากตามข้อกำหนดของใบงาน ช่วยให้ส่ง JSON ของบัญชีได้โดยไม่มีการอ้างอิงวนกลับระหว่างสองฝั่ง

1.4 ขอบเขต Transaction และการทำงานของ COMMIT / ROLLBACK

กำหนด @Transactional ที่เมธอด deposit() ในชั้น Service เพื่อครอบคลุมการค้นหบัญชี่ การเพิ่มยอด และการบันทึกประวัติในธุรกรรมเดียวกัน ถ้าเมธอดทำงานสำเร็จ Spring จะ COMMIT แต่เมื่อ RuntimeException หลุดออกจากเมธอดตามการทดลอง จะ ROLLBACK การเปลี่ยนแปลงของธุรกรรมนั้น การเรียก save() ไม่ได้หมายความว่าธุรกรรมระดับ Service ยืนยันสำเร็จแล้ว การเปลี่ยนแปลงที่เกิดขึ้นยังย้อนกลับได้ก่อน COMMIT หากเอา @Transactional ออกจาก Service การ save() แต่ละครั้งจะจบธุรกรรมของ Repository แยกกัน จึงไม่ย้อนกลับทั้งงานเมื่อเกิด Error ตอนท้าย

1.5 Execution Flow ตั้งแต่ HTTP Request ถึงฐานข้อมูล

1. Postman ส่ง POST /accounts/{id}/deposit พร้อม JSON ที่มี amount
2. AccountController รับ id และ amount แล้วเรียก DepositService.deposit()
3. Spring เปิด Transaction ก่อนเข้าเมธอดผ่าน Service proxy
4. Service ค้นหาบัญชี่ เพิ่ม balance และเรียก AccountRepository.save()
5. Service สร้าง DepositTransaction ตั้ง amount และ account แล้วเรียก DepositRepository.save()
6. ถ้าทำงานครบจะ COMMIT และ Controller ตอบข้อความ Deposit successful หากเกิด RuntimeException ที่ไม่ได้จับไว้จะ ROLLBACK และคำขอล้มเหลว

ส่วนที่ 2 Code และคำอธิบาย

โค้ดต่อไปนี้จะแสดงส่วนสำคัญจากโปรเจกต์ โดยละ package, import และ getter/setter บางส่วนเพื่อให้เห็นโครงสร้างชัดเจน

2.1 Account Entity

```
@Entity
@Table(name = "account")
public class Account {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String accountNumber;
    private String ownerName;
    private Double balance;

    // ละ constructor และ getter/setter
}
```

คำอธิบาย

@Entity ให้ JPA จัดการคลาส Account และ @Table กำหนดชื่อตาราง account ส่วน @Id และ @GeneratedValue กำหนด Primary Key ที่ฐานข้อมูลสร้างให้อัตโนมัติ ฟิลด์ accountNumber, ownerName และ balance เก็บเลขบัญชี ชื่อเจ้าของบัญชี และยอดเงิน โดยใช้ชนิด Double ตามใบงาน

2.2 DepositTransaction Entity

```
@Entity
@Table(name = "deposit_transaction")
public class DepositTransaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Double amount;

    @ManyToOne
    @JoinColumn(name = "account_id")
    private Account account;

    // ละ constructor และ getter/setter
}
```

คำอธิบาย

DepositTransaction เก็บรายการฝากเงิน โดย amount คือจำนวนเงินที่ฝาก และ account อ้างถึงบัญชีเจ้าของรายการ @ManyToOne แสดงว่าหลายรายการฝากอ้างอิงถึงบัญชีเดียวกันได้ ส่วน @JoinColumn กำหนดชื่อคอลัมน์ Foreign Key เป็น account_id

2.3 Repository

```
public interface AccountRepository
    extends JpaRepository<Account, Long> {
}

public interface DepositRepository
    extends JpaRepository<DepositTransaction, Long> {
}
```

แต่ละ interface อยู่คนละไฟล์ และรับความสามารถพื้นฐานจาก JpaRepository เช่น save(), findById() และ findAll() โดย Long เป็นชนิดของ Primary Key ของ Entity ทั้งสอง

2.4 AccountService

```
@Service
public class AccountService {
    private final AccountRepository accountRepository;

    public AccountService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    public Account createAccount(Account form) {
        if (form.getAccountNumber() == null || form.getAccountNumber().isBlank()
            || form.getOwnerName() == null || form.getOwnerName().isBlank()) {
            throw new IllegalArgumentException("Account number and owner name are required");
        }
        if (form.getBalance() == null || !Double.isFinite(form.getBalance()) || form.getBalance() < 0) {
            throw new IllegalArgumentException("Balance must be non-negative");
        }
        Account account = new Account();
        account.setAccountNumber(form.getAccountNumber());
        account.setOwnerName(form.getOwnerName());
        account.setBalance(form.getBalance());
        return accountRepository.save(account);
    }

    public Account getAccountById(Long id) {
        return accountRepository.findById(id)
            .orElseThrow(() -> new NoSuchElementException("Account not found: " + id));
    }
}
```

คำอธิบายการทำงาน

createAccount() ตรวจสอบว่ามีเลขบัญชีและชื่อเจ้าของ และยอดเริ่มต้นไม่ติดลบ จากนั้นสร้าง Account ใหม่และบันทึกลงฐานข้อมูล ส่วน getAccountById() ค้นหาด้วย id และโยน NoSuchElementException เมื่อไม่พบบัญชี เพื่อให้ Controller ตอบ 404

2.5 DepositService และ @Transactional

```
@Service
public class DepositService {
    private final AccountRepository accountRepository;
    private final DepositRepository depositRepository;
    public DepositService(AccountRepository accountRepository, DepositRepository depositRepository) {
        this.accountRepository = accountRepository;
        this.depositRepository = depositRepository;
    }
    @Transactional
    public void deposit(Long accountId, Double amount) {
        if (amount == null || !Double.isFinite(amount) || amount <= 0) {
            throw new IllegalArgumentException("Amount must be greater than zero");
        }
        Account account = accountRepository.findById(accountId)
            .orElseThrow(() -> new NoSuchElementException("Account not found: " + accountId));
        double newBalance = account.getBalance() + amount;
        if (!Double.isFinite(newBalance)) {
            throw new IllegalArgumentException("Balance is too large");
        }
        account.setBalance(newBalance);
        accountRepository.save(account);
        DepositTransaction deposit = new DepositTransaction();
        deposit.setAmount(amount);
        deposit.setAccount(account);
        depositRepository.save(deposit);
    }
}
```

คำอธิบายการทำงาน

deposit() ตรวจสอบเงินให้มากกว่าศูนย์ ค้นหาบัญชี เพื่อยอดและบันทึก Account จากนั้นสร้างและบันทึก DepositTransaction ที่อ้างถึงบัญชีเดิม @Transactional ครอบงำขั้นตอนให้สำเร็จหรือยกเลิกไปด้วยกัน การตรวจสอบ Double.isFinite() ป้องกันค่าตัวเลขที่ไม่ใช่จำนวนจำกัด

สำหรับข้อ 12 ให้เพิ่ม `throw new RuntimeException("Test Rollback");` หลัง `depositRepository.save(deposit)` และในข้อ 13 ให้เอาเฉพาะ `@Transactional` ออกโดยยังเก็บ `throw` ไว้ โค้ดด้านบนเป็นเวอร์ชันทำงานปกติ

2.6 AccountController

```
@RestController
@RequestMapping("/accounts")
public class AccountController {
    private final AccountService accountService;
    private final DepositService depositService;

    public AccountController(AccountService accountService, DepositService depositService) {
        this.accountService = accountService;
        this.depositService = depositService;
    }

    @PostMapping
    public Account createAccount(@RequestBody Account account) {
        return accountService.createAccount(account);
    }

    @GetMapping("/{id}")
    public Account getAccount(@PathVariable Long id) {
        return accountService.getAccountById(id);
    }

    @PostMapping("/{id}/deposit")
    public Map<String, String> deposit(@PathVariable Long id, @RequestBody Map<String, Double>
body) {
        depositService.deposit(id, body.get("amount"));
        return Map.of("message", "Deposit successful");
    }

    // ละเมียดจัดการข้อผิดพลาด
}
```

คำอธิบาย

`@RestController` ส่งข้อมูลตอบกลับเป็น JSON และ `@RequestMapping("/accounts")`

กำหนดเส้นทางหลัก Controller ใช้ `AccountService` สำหรับสร้างและค้นหาบัญชี และใช้ `DepositService`

สำหรับฝากเงิน การสร้างบัญชีและฝากสำเร็จตอบ 200 ตามตัวอย่างในใบงาน ส่วนข้อมูลไม่ถูกต้องตอบ 400 และบัญชีที่ไม่พบตอบ 404

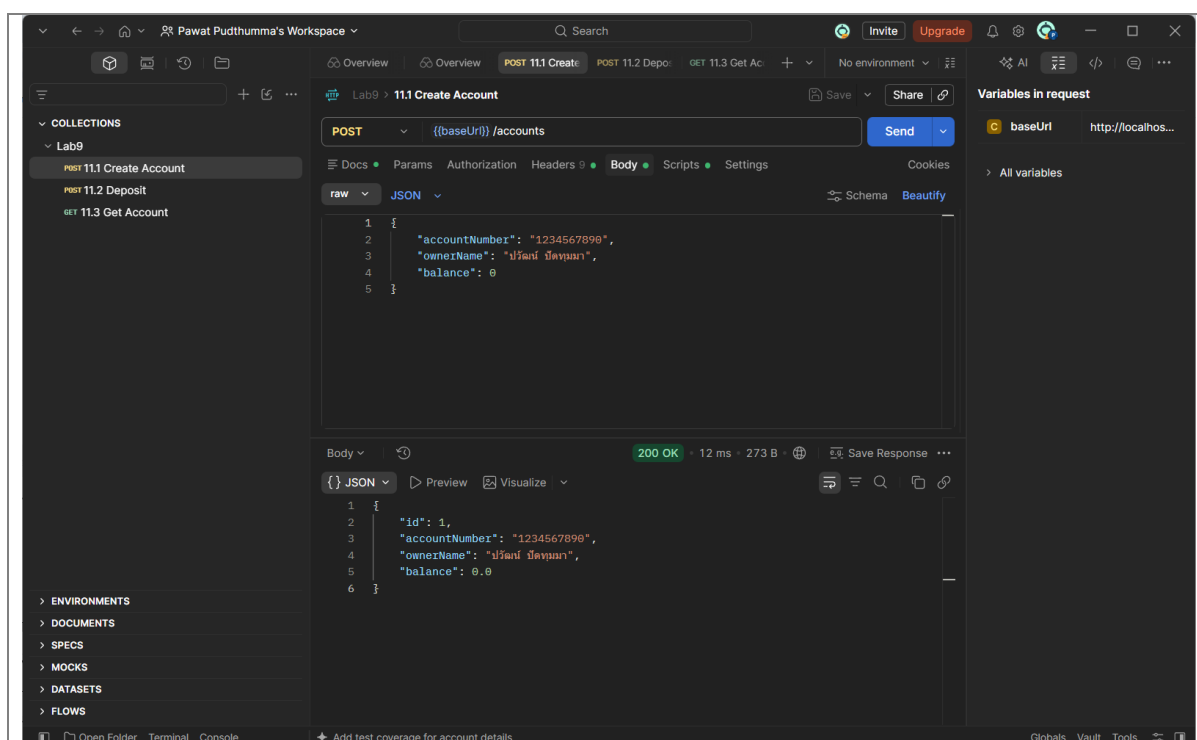
Method	URL	หน้าที่
POST	/accounts	สร้างบัญชี
GET	/accounts/{id}	ดูข้อมูลบัญชี
POST	/accounts/{id}/deposit	ฝากเงิน

ส่วนที่ 3 ภาพหน้าจอและผลการทำงาน

กรอบภาพในส่วนนี้เว้นไว้สำหรับนำภาพการทดลองจริงมาวางตามลำดับ ภาพ Postman ต้องเห็น Request หรือ Response และ HTTP Status ส่วนภาพ pgAdmin ต้องเห็นฐานข้อมูล lab9 และผล Query ที่เกี่ยวข้อง

3.1 การสร้าง Account

อ้างอิงข้อ 11.1 — ส่ง POST `http://localhost:8080/accounts` โดยกำหนด Body เป็น raw → JSON เลขบัญชี 1234567890 ชื่อเจ้าของ ปวีณณ์ ปัดทุมมา และ balance เท่ากับ 0



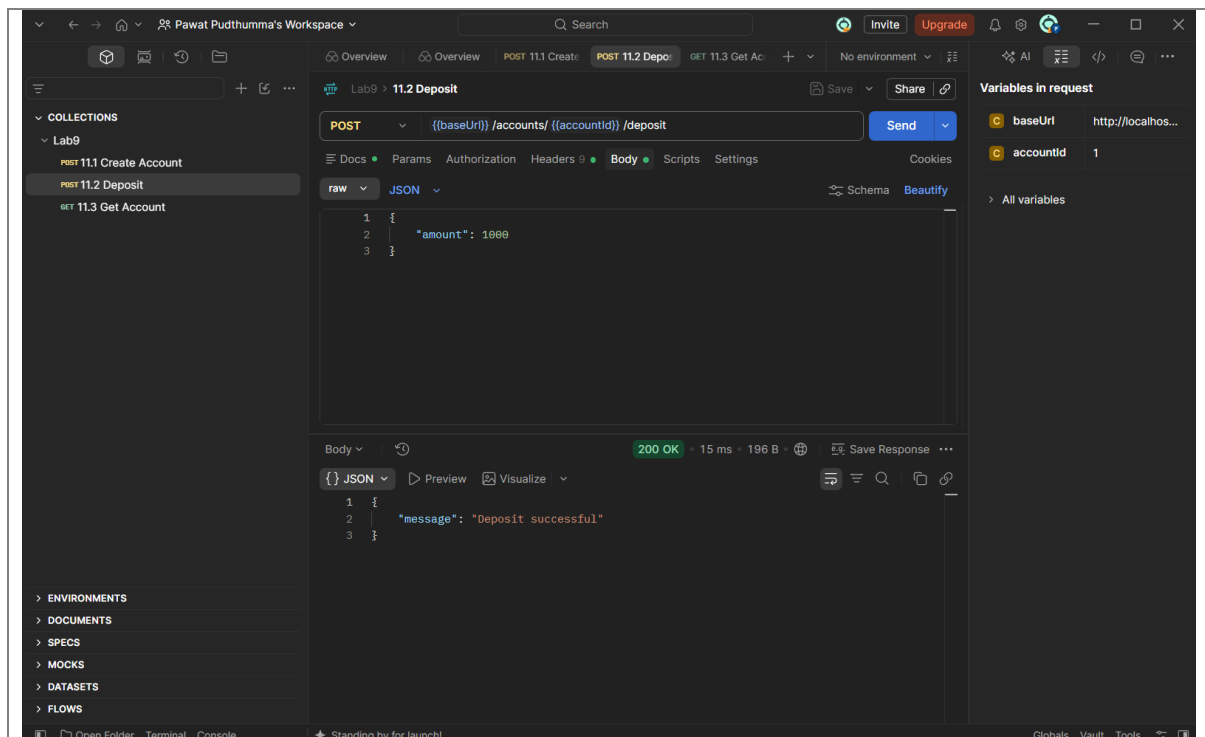
ภาพที่ 1 การสร้างบัญชีธนาคาร

ผลการทดลอง: API ตอบกลับ 200 OK และสร้างบัญชี id = 1 โดยมีชื่อเจ้าของบัญชีเป็น ปวัฒน์ ปัดทุมมา และยอดเงินเริ่มต้น balance = 0.0 แสดงว่าระบบบันทึกบัญชีใหม่ลงฐานข้อมูลสำเร็จ

3.2 การฝากเงินสำเร็จ

อ้างอิงข้อ 11.2 — ส่ง POST `http://localhost:8080/accounts/1/deposit` โดยกำหนด Body เป็น raw → JSON ดังนี้

```
{ "amount": 1000 }
```



ภาพที่ 2 ผลการฝากเงินสำเร็จ

ผลการทดลอง: API ตอบกลับ 200 OK พร้อมข้อความ “Deposit successful” แสดงว่าเมธอด `deposit()`

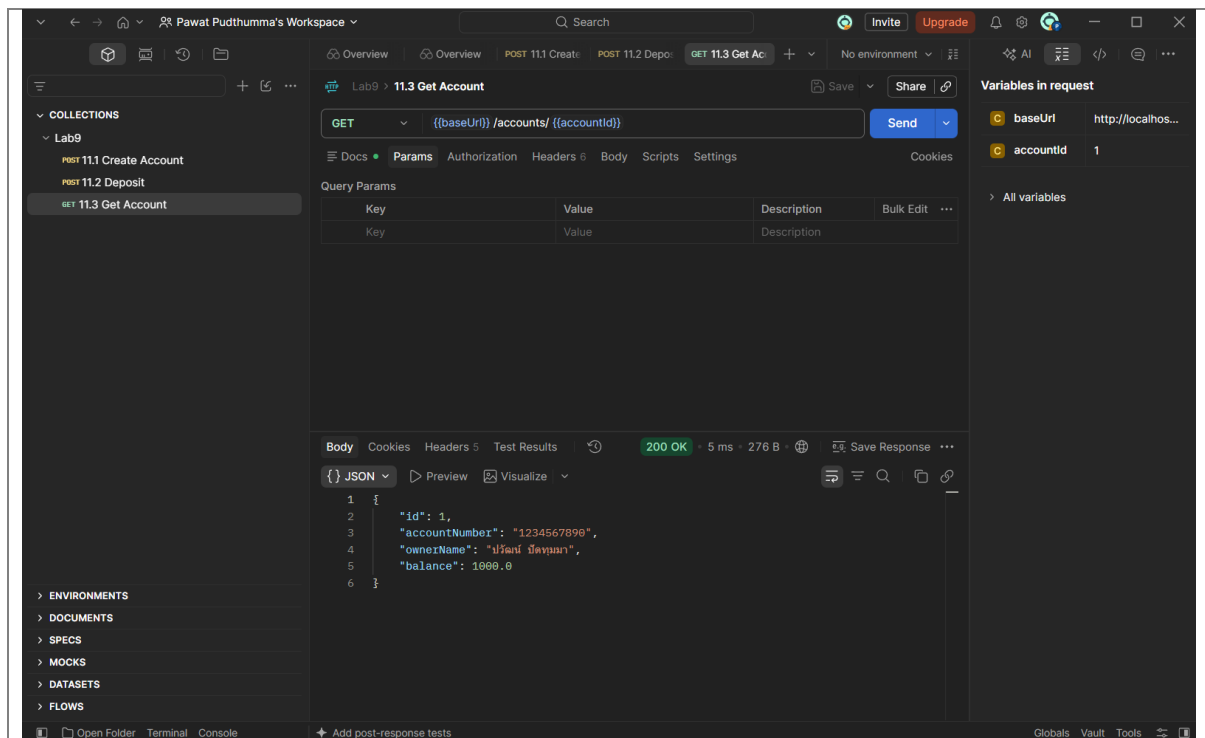
ทำงานเสร็จตามปกติ การทำงานภายใต้ `@Transactional`

รวมการเพิ่มยอดบัญชีและการบันทึกรายการฝากเป็นธุรกรรมเดียวกัน

ตรวจสอบผลการบันทึกจริงจากยอดบัญชีและตารางรายการฝากในหัวข้อ 3.3 และ 3.4

3.3 ตรวจสอบ Account

อ้างอิงข้อ 11.3 — ส่ง GET `http://localhost:8080/accounts/1` โดยไม่ต้องใส่ Body



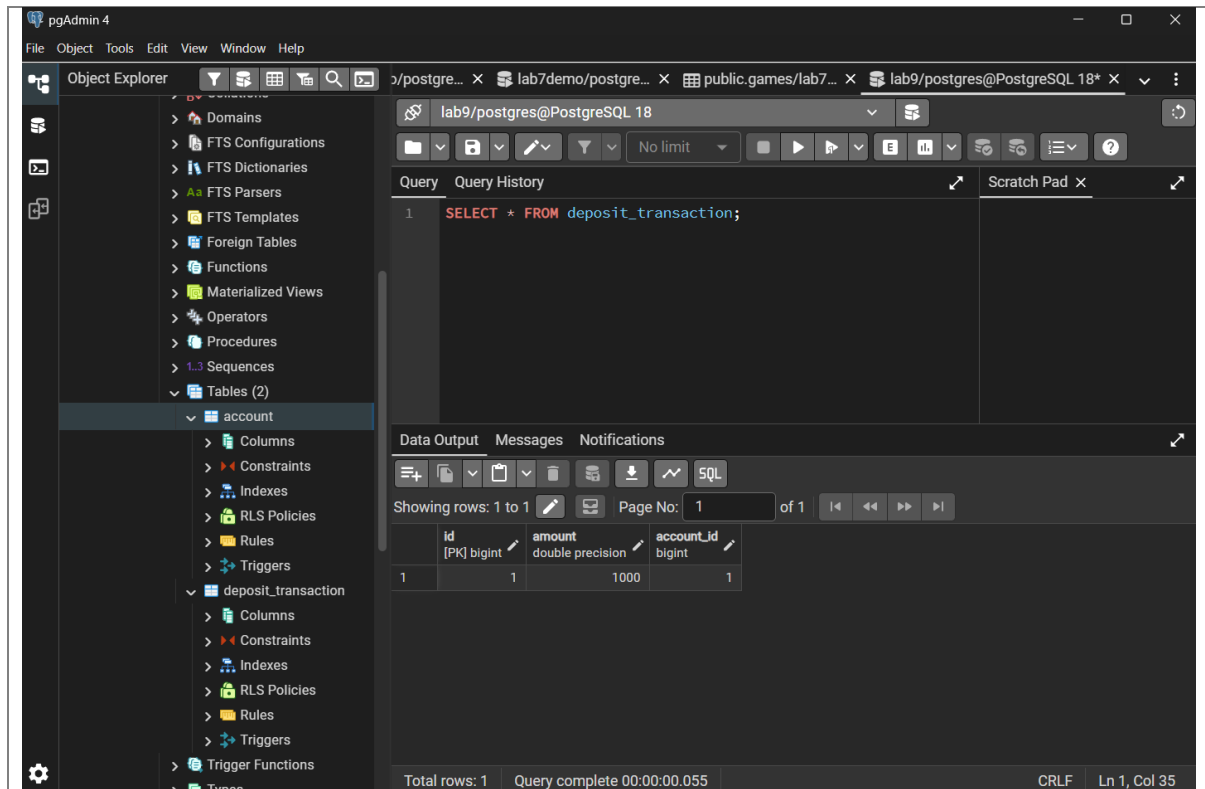
ภาพที่ 3 ข้อมูลบัญชีหลังฝากเงิน

ผลการทดลอง: API ตอบกลับ 200 OK และแสดง balance = 1000.0 จากยอดเดิม 0.0 ยอดเงินเพิ่มขึ้น 1,000 บาทตรงกับจำนวนเงินที่ส่งในคำขอฝากเงิน โดย id และข้อมูลเจ้าของบัญชียังคงเดิม

3.4 ตรวจสอบ DepositTransaction

อ้างอิงข้อ 11.4 — เปิด Query Tool ใน pgAdmin โดยเลือกฐานข้อมูล lab9 แล้วรันคำสั่ง

`SELECT * FROM deposit_transaction;`



ภาพที่ 4 ตาราง deposit_transaction ในฐานข้อมูล lab9

ผลที่ตรวจสอบจากฐานข้อมูล: พบรายการฝาก 1 แถว โดย id = 1, amount = 1000 และ account_id = 1

สอดคล้องกับบัญชีและจำนวนเงินที่ฝาก คอลัมน์ account_id เป็น Foreign Key ที่อ้างอิงถึง account.id

รายการฝากนี้เป็นข้อมูลจากการฝากปกติก่อนเริ่มทดลอง Rollback

3.5 ทดลอง Rollback

อ้างอิงข้อ 12 — ทดลองโดยคอง @Transactional และโยน RuntimeException

หลังบันทึกข้อมูลทั้งสองส่วน

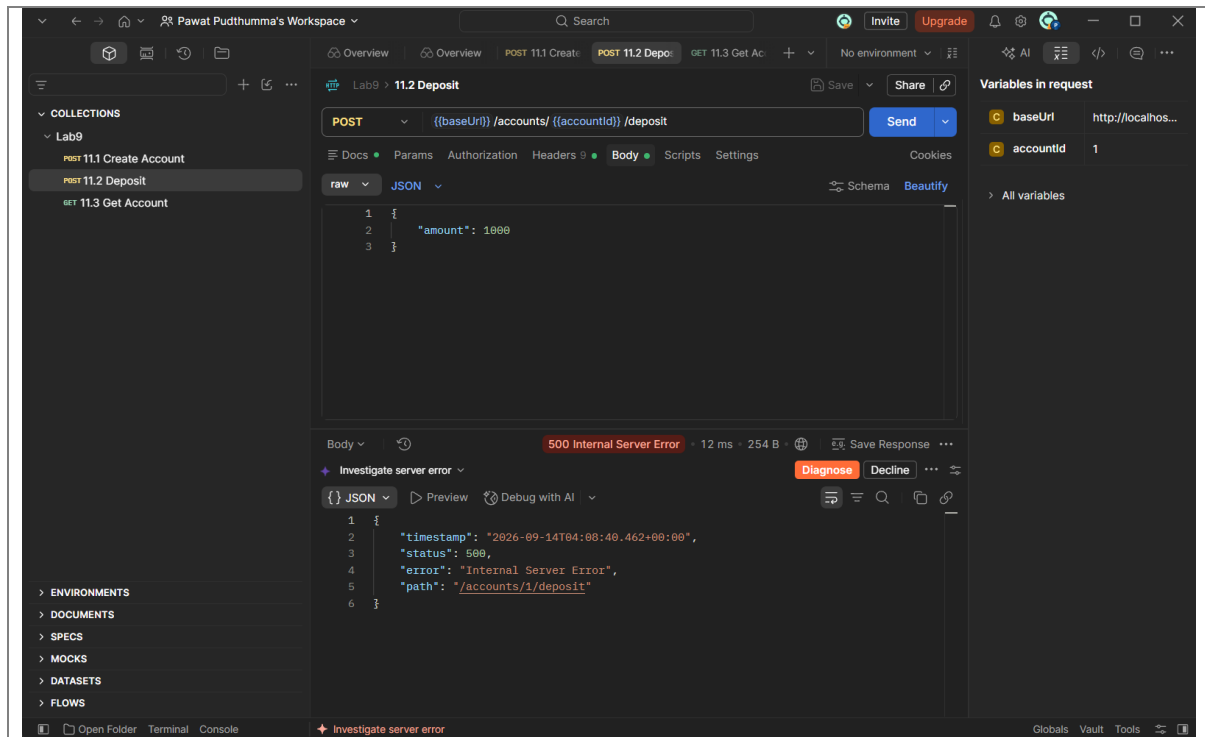
คอง @Transactional ไว้เหนือเมธอด deposit() และเพิ่มคำสั่งต่อไปนี้ท้ายเมธอด หลัง

accountRepository.save(account) และ depositRepository.save(deposit)

แล้วหยุดโปรแกรมและรันใหม่

`throw new RuntimeException("Test Rollback");`

ใช้คำขอฝากเงินเดิม ส่ง amount = 1000 เข้าบัญชี id = 1 อีกครั้ง โดยก่อนทดลองมียอด 1000.0
และรายการฝาก 1 แถว



ภาพที่ 5 การเกิด Error ขณะใช้ @Transactional

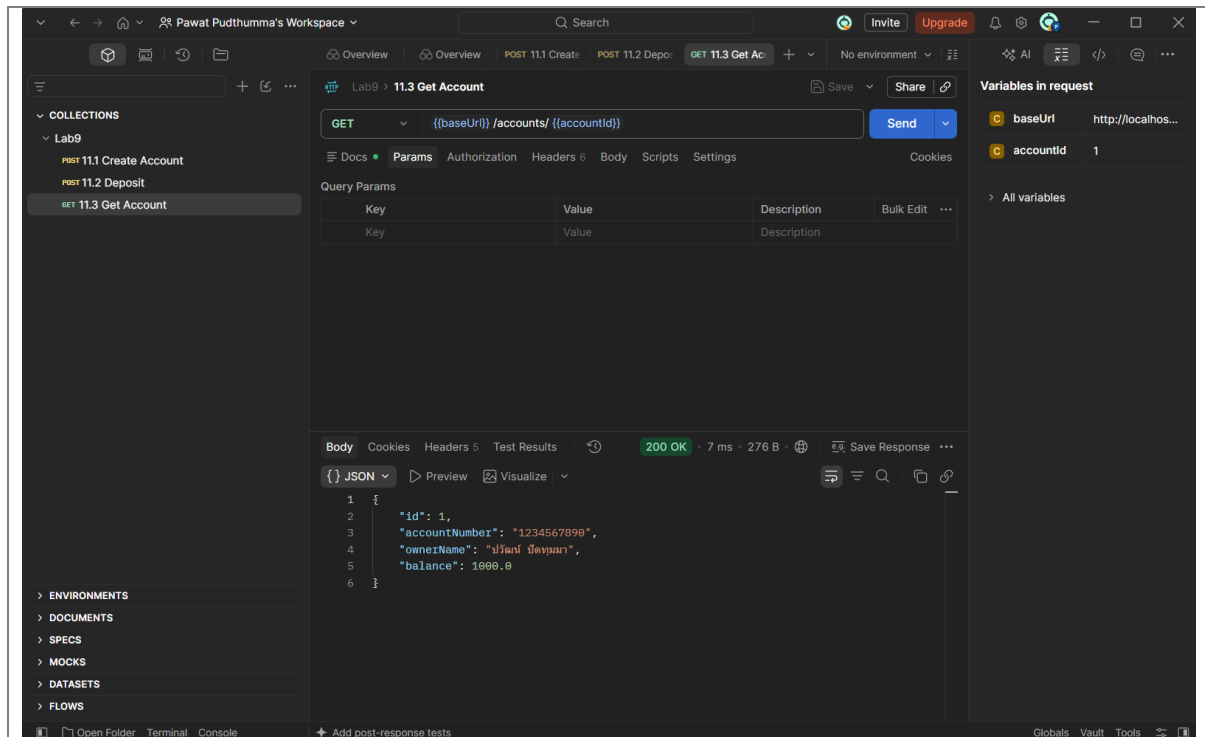
ผลการทดลอง: API ตอบกลับ 500 Internal Server Error เพราะ RuntimeException

หลุดออกจากเมธอดที่มี @Transactional ระบบจึงยกเลิกการเปลี่ยนแปลงของธุรกรรมครั้งนั้น แม้โค้ดจะเรียก save() ทั้งสองครั้งแล้ว

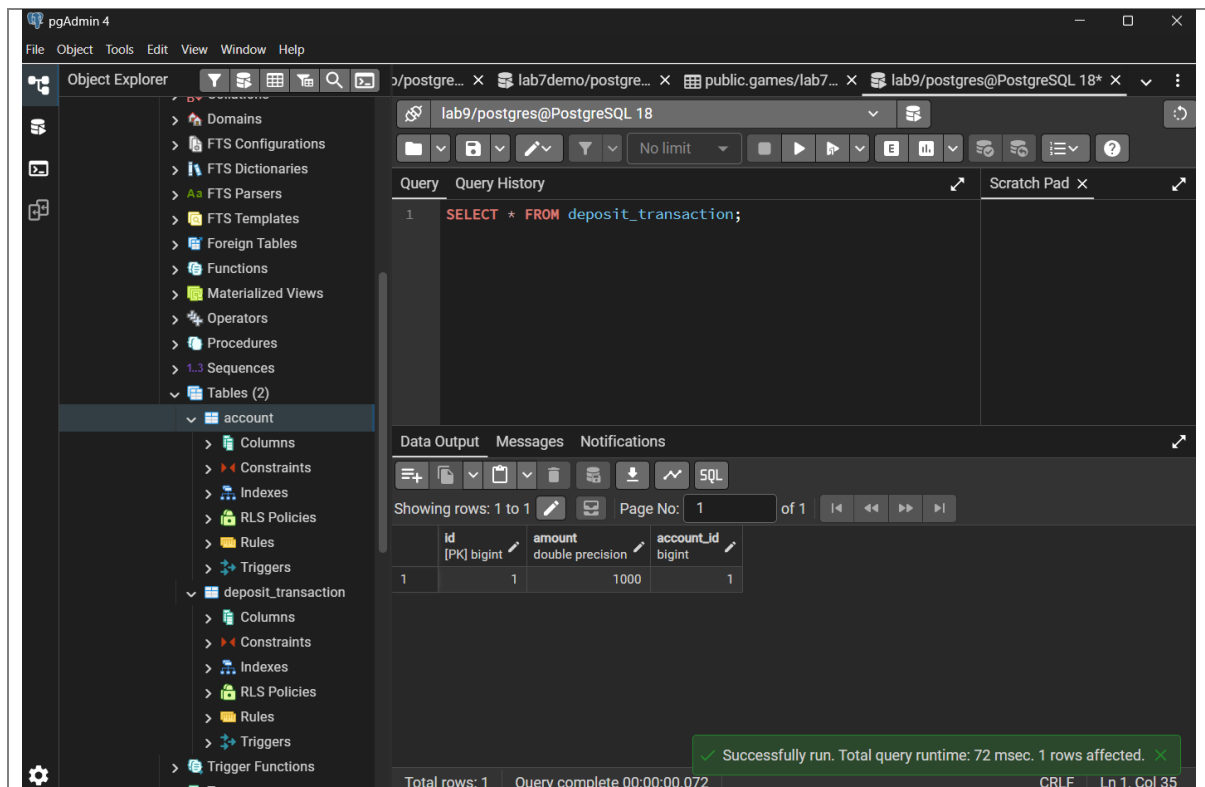
HTTP Status: 500 Internal Server Error; ข้อผิดพลาดในเซิร์ฟเวอร์: java.lang.RuntimeException:

Test Rollback

หลังได้รับ Error ให้ตรวจทั้งยอดบัญชีผ่าน GET /accounts/1 และประวัติใน pgAdmin ด้วย SELECT *
FROM deposit_transaction;



ภาพที่ 6 ตรวจยอดบัญชีหลัง Rollback



ภาพที่ 7 ตรวจสอบประวัติการฝากหลัง Rollback

ผลการตรวจสอบ: ยอดเงินยังเป็น 1000.0 และจำนวนรายการฝากยังเป็น 1 แถว

การฝากที่ล้มเหลวไม่เพิ่มยอดและไม่สร้างประวัติใหม่ เนื่องจากทั้งสองการเปลี่ยนแปลงถูก Rollback ในธุรกรรมเดียวกัน

ผลจริง: balance ก่อน/หลัง = 1000.0 / 1000.0; จำนวนแถวก่อน/หลัง = 1 / 1

ในการทดลองขั้นถัดไป คงบรรทัด `throw` ไว้แล้วปิด `@Transactional` เพื่อเปรียบเทียบผล

3.6 เปรียบเทียบมีและไม่มี `@Transactional`

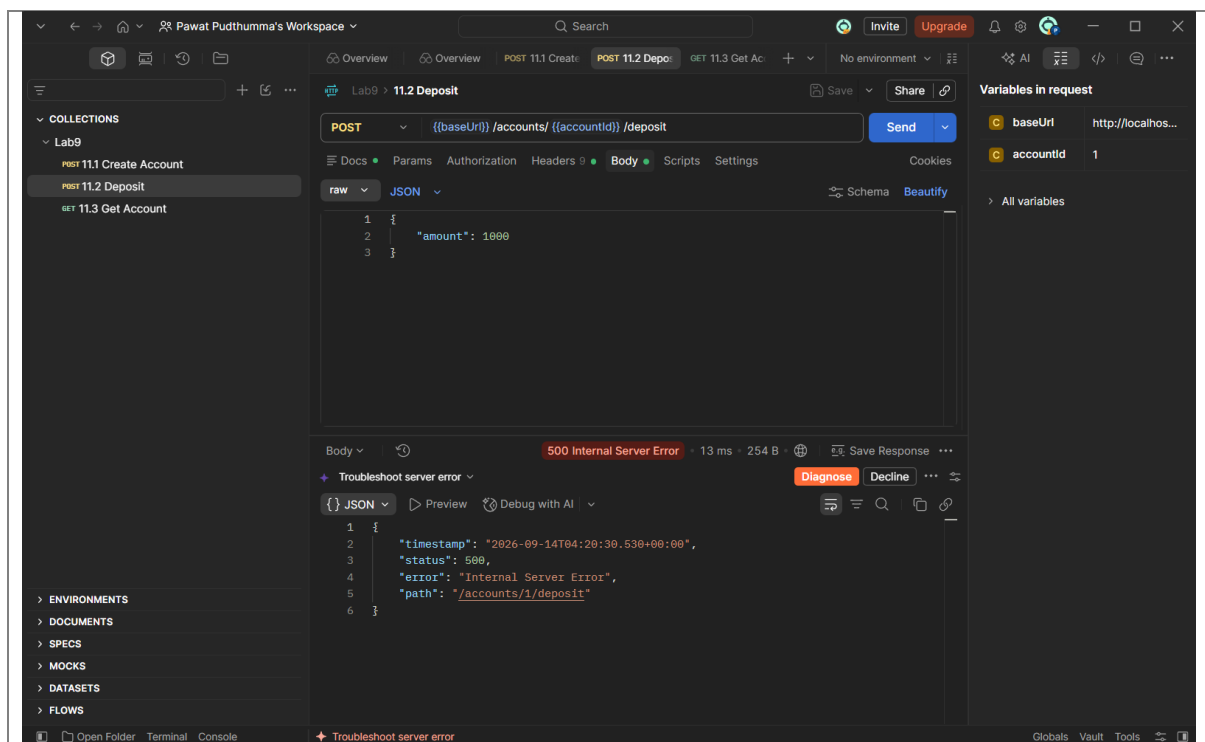
อ้างอิงข้อ 13 — ทดลองโดยปิด `@Transactional` และยังคง `RuntimeException` ไว้ท้ายเมธอด

Comment บรรทัด `@Transactional` เป็น `//@Transactional` โดยยังคง `throw new`

`RuntimeException("Test Rollback");` ไว้ท้ายเมธอด จากนั้นหยุดโปรแกรมและรันใหม่

ในการทดลองช่วงนี้ส่งคำขอฝากเงิน `amount = 1000` เข้า รวมเกิดรายการฝากใหม่ 3 รายการ

จากยอดเริ่มต้น 1000.0 และประวัติเดิม 1 แถว



ภาพที่ 8 การเกิด Error เมื่อไม่มี `@Transactional` ที่เมธอดฝากเงิน

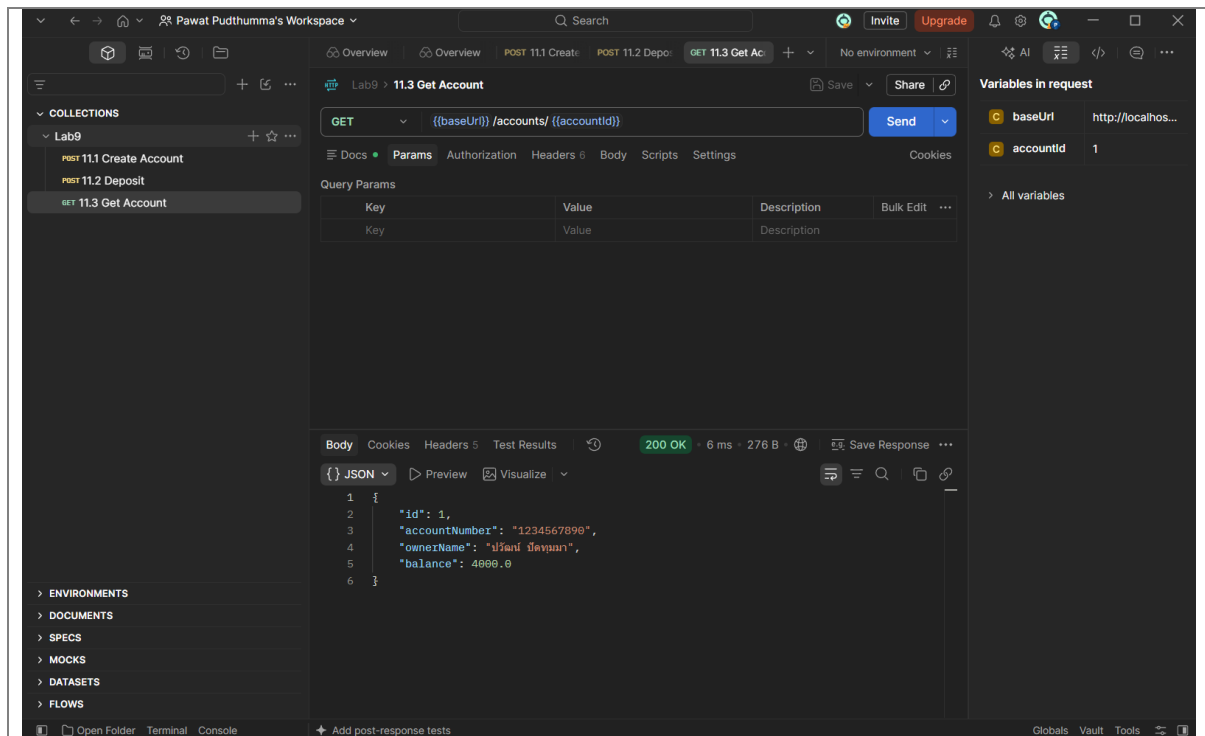
ผลการทดลอง: API เกิด Error จาก RuntimeException แต่การบันทึกผ่าน Repository

แต่แต่ละครั้งเสร็จสิ้นไปแล้วก่อนเกิด Error จึงไม่มีธุรกรรมระดับ Service

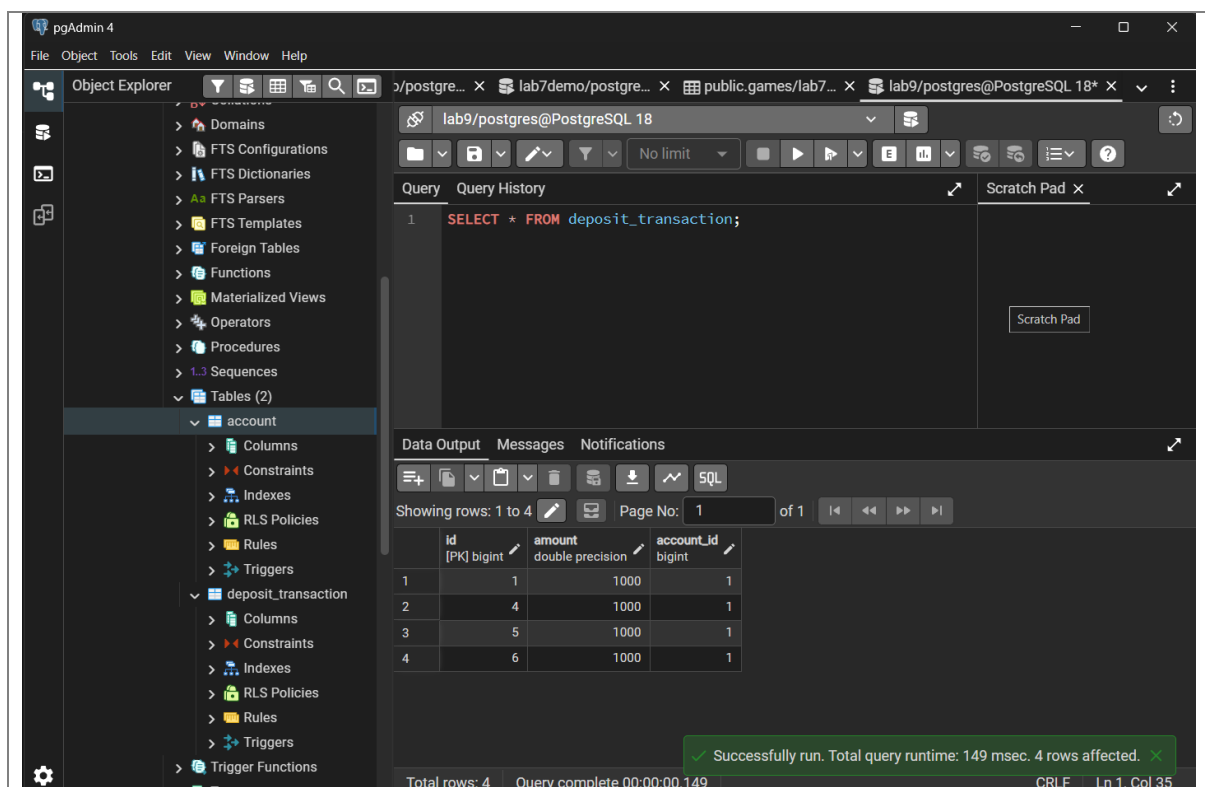
ที่ย้อนการเปลี่ยนแปลงทั้งสองอย่างกลับพร้อมกัน

ข้อผิดพลาดที่เกิดขึ้น: 500 Internal Server Error จาก RuntimeException("Test Rollback");

การตรวจบัญชีด้วย GET ภายหลังตอบกลับ 200 OK



ภาพที่ 9 ยอดบัญชีเมื่อไม่มี Transaction ครอบทั้งเมธอด



ภาพที่ 10 ประวัติการฝากเมื่อไม่มี Transaction ครอบทั้งเมธอด

ตารางเปรียบเทียบผลการทดลองจริง (ช่วงไม่มี annotation มีการฝากซ้ำ 3 ครั้ง)

กรณี	HTTP	ยอดเงิน	รายการฝาก
ฝากปกติ	200	1000.0	1 แถว
มี annotation + Error	500	1000.0	1 แถว
ไม่มี annotation + Error (ฝากซ้ำ 3 ครั้ง)	500	4000.0	4 แถว

ผลจริงเมื่อไม่มี @Transactional: balance ก่อน/หลัง = 1000.0 / 4000.0; จำนวนรายการฝากก่อน/หลัง =

1 / 4 โดยรายการปัจจุบันมี id = 1, 4, 5, 6 และ amount = 1000 ทุกแถว ยอดรวมรายการฝากตรงกับ

balance = 4000.0 ตัวเลข id ที่ซ้ำกันได้เกิดจาก sequence ซึ่งไม่ย้อนกลับตามธุรกรรมที่ Rollback

สรุปผล: HTTP 500 ไม่ได้ยืนยันว่า Rollback แล้ว ต้องตรวจทั้งยอดบัญชีและรายการฝากประกอบกัน

หากส่งเพียงครั้งเดียวตามใบงานจะได้ 2000.0 และ 2 แถว แต่ผลจริงรอบนี้เป็น 4000.0 และ 4 แถว

เพราะมีการฝากซ้ำในช่วงไม่มี @Transactional

ส่วนที่ 4 คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

กำหนดขอบเขตธุรกรรมให้หลายคำสั่งทำงานเป็นหน่วยเดียวกัน ถ้าทำงานสำเร็จจะ COMMIT แต่เมื่อเกิด RuntimeException ตามการทดลองและไม่ได้จับไว้ภายในเมธอด Spring จะ ROLLBACK การเปลี่ยนแปลงในธุรกรรมนั้น

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงินต้องเพิ่มยอดบัญชีและบันทึกประวัติให้สำเร็จพร้อมกัน ถ้าสำเร็จเพียงบางส่วน ข้อมูลยอดเงินและประวัติจะไม่ตรงกัน Transaction ช่วยยกเลิกทั้งสองการเปลี่ยนแปลงเมื่อธุรกรรมล้มเหลว

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันการเปลี่ยนแปลงของธุรกรรมให้เป็นข้อมูลที่บันทึกสำเร็จ ส่วน ROLLBACK คือการยกเลิกการเปลี่ยนแปลงในธุรกรรมที่ยังไม่ COMMIT โดยไม่ลบข้อมูลจากธุรกรรมก่อนหน้าสำเร็จแล้ว

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

เป็น One-to-Many จาก Account ไปยัง DepositTransaction คือหนึ่งบัญชีมีรายการฝากได้หลายรายการ ในโค้ดกำหนด @ManyToOne ที่ DepositTransaction และใช้ account_id อ้างถึง account.id โดย Account ไม่ต้องมีลิสต์รายการฝากตามขอบเขตของแอ็บ

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

สำหรับ RuntimeException ที่โยนท้ายเมธอดในแอ็บนี้

ข้อมูลที่แก้ไขในธุรกรรมการฝากครั้งนั้นจะถูกย้อนกลับ

ยอดเงินและจำนวนรายการฝากจึงคงเท่ากับก่อนเริ่มฝากครั้งที่ล้มเหลว ทั้งนี้กฎ Rollback

อาจต่างกันหากเปลี่ยนชนิด Exception หรือการตั้งค่า

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อมี @Transactional การเกิด RuntimeException ทำให้ยอดคงอยู่ที่ 1000.0 และมีรายการฝาก 1 แถว เมื่อปิด annotation แม้เกิด Error การ save() แต่ครั้งบันทึกสำเร็จไปก่อนแล้ว จึงยังเพิ่มยอดและประวัติ ในการทดลองนี้เกิดการฝากซ้ำ 3 ครั้ง ครั้งละ 1000 ทำให้ยอดสุดท้ายเป็น 4000.0 และมีรายการฝากรวม 4 แถว แสดงว่าไม่มีการ Rollback ครบทั้งเมธอด Service

หลังทดลองเสร็จ: ลบ throw new RuntimeException("Test Rollback"); และใส่ @Transactional กลับคืนแล้ว พร้อม build และรันโปรแกรมใหม่ ทดสอบอัตโนมัติผ่านครบ 4 กรณีบนฐานข้อมูล H2 แยกจาก PostgreSQL โดยคงข้อมูลผลทดลองไว้ที่ยอด 4000.0 และรายการฝาก 4 แถว